Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54820063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Experiences and Challenges in AI-Driven Modular Software Development Using Large Language Models for Code Generation

Sadi Evren SEKER[1]

[1]Istanbul Universitesi

October 12, 2024

Abstract

The development of a modular software platform using large language models (LLMs) for code generation presents unique opportunities and challenges. This paper explores the experiences and difficulties encountered during the creation of an AI-driven business intelligence platform built with LLM assistance. By using an iterative approach to generate modular code, the project aimed to accelerate development and automate routine tasks. However, challenges such as inconsistency in the generated code, hallucinations, lack of long-term memory, and integration complexities emerged. These limitations necessitated manual intervention for code refinement, debugging, and integration to ensure project-wide consistency. The study discusses strategies to address these issues, including structured prompting, automated testing, and iterative refinement. The findings reveal that while LLMs significantly reduce development time and facilitate rapid prototyping, they are not a complete substitute for human expertise. The paper offers practical insights into optimizing the use of LLMs in software engineering, demonstrating both the potential benefits and current limitations of AI-assisted code generation in modular software projects.

Experiences and Challenges in AI-Driven Modular Software Development Using Large Language Models for Code Generation

Sadi Evren SEKER11* Yazar için dipnot metnini (varsa) buraya yerleştirin.

Istanbul University, Faculty of Computer and Information Technologies, Department of Computer Engineering, academic@sadievrenseker.com

Abstract

The development of a modular software platform using large language models (LLMs) for code generation presents unique opportunities and challenges. This paper explores the experiences and difficulties encountered during the creation of an AI-driven business intelligence platform built with LLM assistance. By using an iterative approach to generate modular code, the project aimed to accelerate development and automate routine tasks. However, challenges such as inconsistency in the generated code, hallucinations, lack of long-term memory, and integration complexities emerged. These limitations necessitated manual intervention for code refinement, debugging, and integration to ensure project-wide consistency. The study discusses strategies to address these issues, including structured prompting, automated testing, and iterative refinement. The findings reveal that while LLMs significantly reduce development time and facilitate rapid prototyping, they are not a complete substitute for human expertise. The paper offers practical insights into optimizing the use of LLMs in software engineering, demonstrating both the potential benefits and current limitations of AI-assisted code generation in modular software projects.

Keywords: Large Language Models (LLMs), AI-Driven Code Generation, Modular Software Development, Integration Challenges, Hallucination in LLMs

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Introduction

The use of large language models (LLMs) in software development has gained considerable attention as a means of automating code generation, accelerating development processes, and reducing the manual effort required for coding. LLMs such as ChatGPT have shown potential in generating code snippets, providing documentation assistance, and even helping with software design. However, while LLMs offer promising capabilities, their practical application in real-world projects presents unique challenges that require more in-depth exploration and evaluation. This paper examines the experiences and challenges encountered during the creation of a modular software platform developed with the assistance of LLMs for code generation. The project employed a modular approach, where each functionality was developed as a separate module using an iterative process of LLM-generated code and human refinement. While this method aimed to facilitate rapid prototyping, automate routine coding tasks, and enable flexible integration of new features, several issues emerged. These included code inconsistency, hallucination problems where LLMs generated plausible yet incorrect code, the lack of long-term memory and context retention, and integration complexities when combining multiple modules into a cohesive system. The findings of this study are significant for the literature as they provide a practical perspective on the limitations and benefits of using LLMs in software engineering. Although research on AI-driven code generation has primarily focused on isolated code snippets or theoretical approaches, there has been limited exploration of LLMs' effectiveness in end-to-end software development. This paper fills that gap by discussing real-world challenges and strategies for using LLMs in a modular, iterative development context, highlighting the need for structured prompting, validation, and human oversight to ensure quality and consistency. The insights gained from this study are crucial for guiding future research and practical applications of LLMs in software development. By documenting the specific challenges related to hallucinations, memory limitations, and modular integration, this paper provides a foundation for developing improved prompting techniques, context-aware LLMs, and hybrid approaches that combine AI-generated code with human expertise. The strategies proposed for addressing these issues, such as structured prompts, automated testing, and iterative refinement, can serve as best practices for developers seeking to integrate LLMs into their workflows. Moreover, the findings can inform future studies that aim to enhance LLM capabilities in software engineering. The documented challenges related to context retention and hallucination suggest potential directions for research on extending LLM memory and improving their accuracy in understanding complex, project-wide requirements. Additionally, the practical experiences discussed in this paper may inspire further exploration of hybrid development models where AI-generated code complements human decision-making, leading to more efficient and reliable software development processes. In conclusion, this paper not only contributes to the literature by providing a detailed account of the practical challenges in AI-assisted software development but also offers valuable insights and recommendations for improving the use of LLMs in modular software projects. The findings underscore the need for a nuanced approach to integrating LLMs into software engineering, one that recognizes their potential benefits while addressing their current limitations. Kolay.ai, an AI-powered business intelligence platform designed to help small and medium-sized enterprises (SMEs) improve financial management, customer segmentation, and data visualization, was developed using a unique AI-assisted approach. The project utilized ChatGPT to generate modular code for the Django framework, with each module tailored to specific functionalities such as sales prediction, customer recommendations, and financial forecasting. This paper provides an in-depth analysis of the experiences and challenges encountered during the creation of Kolay.ai using LLM-generated code. It discusses the benefits of using LLMs for modular development, the issues faced in maintaining consistency and quality across modules, and the manual interventions required to integrate LLM-generated code into a cohesive project. The findings contribute to the academic discourse on AI-assisted software development and offer practical insights for developers seeking to leverage LLMs in building complex software systems.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Literature Review

The application of large language models (LLMs) in software development is an emerging area that has garnered significant interest, particularly in the field of AI-driven code generation. While traditional software engineering methods rely heavily on human expertise for writing and maintaining code, the advent of LLMs has introduced new possibilities for automating various aspects of the development process, including code generation, documentation, and testing. The existing literature primarily focuses on LLMs' ability to generate isolated code snippets, automate repetitive coding tasks, and assist with debugging, offering promising results in accelerating development timelines. However, the integration of LLMs into end-to-end software development, especially in modular architectures, remains underexplored. This gap in the literature calls for a closer examination of how LLMs perform in real-world, complex projects that require consistency, context awareness, and integration of multiple components. This review aims to provide a comprehensive overview of the current state of AI-assisted code generation, the benefits and challenges associated with using LLMs for modular software development, and the specific difficulties in applying LLMs to create comprehensive, cohesive software systems. The findings presented in this paper build on this body of research by offering practical insights into the use of LLMs for modular software development and suggesting new directions for future studies.

AI-Assisted Code Generation

AI-assisted code generation has gained attention as a means of accelerating software development and reducing the manual effort involved in writing code. Tools such as Copilot, ChatGPT, and Codex provide developers with automated code suggestions, documentation generation, and assistance in debugging (Chen et al., 2021). These LLMs are trained on vast amounts of code, enabling them to produce code snippets in various programming languages, including Python and frameworks like Django. However, the practical application of LLMs in real-world projects is still evolving. While LLMs can effectively generate code for routine tasks, they may struggle with complex requirements or context-specific problems (Zhang et al., 2023). The limitations of LLMs in understanding the broader context, handling intricate software design patterns, and ensuring code quality remain significant challenges. AI-assisted code generation is rapidly transforming the software development landscape. By leveraging machine learning (ML) and natural language processing (NLP), these tools can analyze code, understand developer intent, and generate code snippets, entire functions, or even complete programs. This technology promises to increase developer productivity, reduce errors, and lower the barrier to entry for aspiring programmers. There are several key approaches and techniques drive AI-assisted code generation and you can find some of the as below list:**Machine Learning on Code:** Large language models (LLMs) are trained on massive code datasets to learn coding patterns, syntax, and best practices. This allows them to generate code that is syntactically correct and semantically meaningful (1, 2).**Natural Language Processing:** NLP techniques enable these tools to understand natural language descriptions of desired functionality and translate them into code (3). This allows developers to express their intent in plain English, making the process more intuitive.**Code Completion and Suggestion:** AI tools can predict the next token or line of code based on the current context, significantly speeding up the coding process and reducing errors (4).**Code Summarization and Documentation:** AI can analyze code and generate concise summaries and documentation, aiding in code comprehension and maintenance (5).**Code Translation:** AI can translate code from one programming language to another, facilitating code migration and modernization (6).

Current Technologies and Comparison

The market is witnessing a surge in AI-powered code generation tools. In Table 1, there is a comparison of some leading technologies:

Table 1: Comparison of Current AI Code Generation Technologies and Tools

Tool	Key Features	Strengths
GitHub Copilot	AI pair programmer, code completion, suggestions	Integratio
ChatGPT	Code generation from natural language prompts, code explanation, code translation	Versatile a
Tabnine	Code completion, whole-line and full-function code generation	Lightweigh
Amazon CodeWhisperer	Code generation, security scanning	Integratio
Google Codey	Code generation, code chat	Strong pe
Replit Ghostwriter	Code completion, code generation, debugging assistance	Integrated

Modular Software Development and Django Framework

Modular software development promotes the creation of software systems as a collection of interchangeable modules, each responsible for a specific functionality (Parnas, 1972). This approach facilitates code reusability, maintainability, and scalability. The Django framework, known for its modular architecture, is widely used for developing web applications due to its "batteries-included" philosophy and support for reusable components (Holovaty & Kaplan-Moss, 2009).

Combining modular development with AI-generated code presents an opportunity to streamline software creation by automating repetitive coding tasks while allowing developers to focus on higher-level design and integration.

Modular software generation with Django is particularly important for the use of LLMs because it aligns well with the capabilities and limitations of current AI-driven code generation. Django, a high-level web framework, supports modular development through its built-in architecture that allows developers to separate different functionalities into independent modules, such as apps, models, views, and templates. This modular approach provides a structured way to develop software incrementally, where each module can be developed, tested, and refined independently before being integrated into the overall system.

Using a modular architecture with Django when employing LLMs for code generation offers several key benefits:

Isolation of Complexity : Modular development enables complex projects to be broken down into smaller, manageable components. Each module addresses a specific functionality (e.g., user authentication, data visualization, or sales forecasting), allowing the LLM to generate code for these discrete parts separately. This approach reduces the cognitive load on the LLM by focusing on specific tasks, leading to better quality output for each module. It also makes it easier to identify and correct issues since problems can be isolated within individual modules without affecting the entire system (Parnas, 1972).

Iterative Development and Refinement: In a modular framework like Django, iterative development becomes more feasible. Developers can prompt LLMs to generate code for individual modules, review the output, and refine it before moving on to the next module. This process allows for a step-by-step approach to building a complete system while continuously improving each component. Iterative refinement is crucial when working with LLMs, as it allows developers to mitigate issues such as hallucinations and ensure code consistency across the project (Larman & Basili, 2003).

Improved Code Reusability and Maintainability: Modular code generated for Django projects can be reused across different parts of the application or even in future projects. When LLMs generate code for Django apps, models, or views, these components can be easily modified and integrated with other modules, leading to a more maintainable codebase. The modular structure also facilitates updates and maintenance because individual modules can be changed without affecting the rest of the system (Gamma, Helm, Johnson, & Vlissides, 1995).

Alignment with LLM Capabilities and Limitations: Current LLMs, including ChatGPT, have limitations when it comes to handling complex projects with multiple interdependent components. Modular development helps to mitigate these limitations by focusing LLM outputs on specific, smaller tasks (Ahmad,

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Zhang, & Liu, 2022). Since LLMs can only process a certain amount of information in a single prompt, generating code for a single Django module allows developers to stay within these constraints while still achieving meaningful progress in the overall project.

Facilitating Testing and Quality Assurance: In modular software generation, each module can be independently tested for functionality and performance. Django's built-in support for testing aligns well with the modular approach, allowing LLM-generated code to be validated using unit tests for individual components. This modular testing approach helps catch issues early in the development process, ensuring that the final integrated system meets the desired quality standards (Pressman, 2014).

Overall, modular software generation with Django provides a structured and manageable way to utilize LLMs for code generation, allowing developers to overcome some of the challenges associated with AI-driven development while taking advantage of Django's modular architecture to build scalable and maintainable software systems.

Challenges and Metricss of Using LLMs in Software Development

While LLMs offer significant potential for automating coding tasks, several challenges persist. Issues such as maintaining code consistency across different modules, understanding project-specific contexts, managing dependencies, and addressing quality concerns often necessitate human intervention (Ahmad et al., 2022). The reliability of LLM-generated code can vary based on the quality of the prompt provided and the complexity of the code being generated, making it important to validate and refine the output.

Table 2: Comparison of Metrics for LLM Code Generations

Metric Category	Metric	Description
Intrinsic Evaluation	Perplexity	Measures how well the model predicts the next token in a sequence. Lov
	Cross-entropy loss	Measures the difference between predicted and true probability distribut
	Code Coverage	Measures the percentage of code paths, branches, or statements execute
	Token Accuracy	Measures the percentage of correct tokens in the generated code compar
Extrinsic Evaluation	Accuracy	Percentage of correct answers on a specific task (e.g., question answering
	BLEU score	Measures similarity between generated text and reference translations.
	ROUGE score	Measures overlap between generated summaries and reference summarie
	Functional Correctness	Measures whether the generated code produces correct outputs when ex
	Compilation Rate	Percentage of generated code snippets that compile successfully.
Human Evaluation	Fluency	Human judges rate the grammatical correctness and naturalness of gene
	Coherence	Judges assess the logical flow and consistency of generated text.
	Relevance	Judges evaluate whether generated text is relevant to given prompt or c
	Code Readability	Judges assess the readability and maintainability of generated code.
	Semantic Correctness	Judges evaluate whether the code implements the intended functionality

The evaluation of large language model (LLM)-generated code involves multiple dimensions to comprehensively assess its quality and performance. The metrics can be categorized into intrinsic evaluation, extrinsic evaluation, and human evaluation, each with specific roles in measuring different aspects of code generation.

Intrinsic evaluation metrics assess the LLM's performance based on statistical properties of the generated text, without executing the code. Common metrics include perplexity, which measures the model's ability to predict the next token in a sequence, and cross-entropy loss, which captures the difference between predicted and actual probability distributions. While these metrics are easy to calculate, they do not necessarily reflect the quality or functionality of the generated code. Metrics such as token accuracy and code coverage provide additional insights into syntactic correctness and the extent to which the code is tested.

Extrinsic evaluation metrics involve executing the generated code to assess its functionality and useful-

ness in real-world tasks. Accuracy on specific tasks, such as classification or question answering, can be measured, as well as task-specific scores like BLEU, ROUGE, and METEOR for language-based tasks. For code generation, functional correctness is a crucial metric that evaluates whether the generated code passes predefined test cases, while compilation rate indicates the percentage of syntactically valid code snippets. These metrics are essential for assessing the practicality of LLM-generated code.

Human evaluation plays a critical role in assessing qualities that are difficult to measure automatically, such as code readability, semantic correctness, fluency, coherence, and relevance. Judges review the generated code for aspects like logical flow, grammatical correctness, and relevance to the problem prompt. Although human evaluation provides valuable insights, it is subjective, time-consuming, and requires domain expertise. The inclusion of metrics such as semantic correctness and code readability helps address real-world concerns about maintainability and long-term usability.

Difficulties of LLMs for Code Generation and ExperIences

While large language models (LLMs) such as ChatGPT offer significant potential in automating code generation and accelerating software development, several inherent difficulties were encountered during the Kolay.ai project. These challenges highlight the limitations of current LLMs and underscore the need for human oversight and iterative refinement in AI-assisted development. The main difficulties experienced with LLM-generated code in the Kolay.ai project includes below topics:

Inconsistency in Generated Code

One of the most common challenges encountered when using LLMs for code generation is the inconsistency in the generated code. Despite providing detailed prompts, the output varied significantly in terms of coding style, naming conventions, and structure. For example, variable names and function signatures generated by ChatGPT often did not follow a consistent pattern across different modules, leading to discrepancies that required manual standardization. This inconsistency could create integration challenges, as mismatched naming conventions and structures resulted in difficulties when combining different modules into a cohesive system. Ensuring consistency across all generated code thus necessitated additional effort to manually review, refactor, and standardize the output.

Lack of Context Awareness

LLMs, by design, have limited context awareness, which is a significant drawback when generating code for complex projects. While they can generate useful code snippets based on a given prompt, they often struggle to understand the broader context of a project. For example, in the Kolay.ai project, the generated code did not always align with the established architecture or existing codebase, as the LLM lacked awareness of the interdependencies between different modules. This limitation made it challenging to ensure the generated code would seamlessly integrate with existing functionalities. As a result, manual intervention was often required to adapt the LLM-generated code to fit the project context and resolve issues related to dependencies, data flow, and shared resources.

Handling Complex Logic and Edge Cases

While LLMs are effective at generating code for straightforward tasks, they often struggle with complex logic or handling edge cases. In the development of Kolay.ai, certain modules required intricate business logic or customized algorithms that could not be fully captured by the LLM. For example, implementing advanced customer segmentation or financial forecasting algorithms required more than just a generic code snippet. The LLM-generated code often lacked the depth needed to handle all the required conditions or edge cases, necessitating additional manual coding to ensure comprehensive coverage. This limitation made it clear that

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

while LLMs can aid in the development process, they are not yet capable of fully replacing human expertise in crafting complex software solutions.

Integration Complexity

The modular approach adopted in the Kolay.ai project aimed to simplify the integration process by isolating functionalities within separate modules. However, integrating these LLM-generated modules into a unified system still posed significant challenges. Issues arose when multiple modules shared resources, such as database connections, middleware configurations, or shared data models. The LLM-generated code often did not account for the global architecture of the project, leading to conflicts when combining modules. Resolving these conflicts required manual intervention to ensure that shared resources were managed properly, dependencies were correctly established, and global configurations were consistently applied across all modules.

Quality Assurance and Debugging

Ensuring the quality of LLM-generated code remained a substantial challenge throughout the development of Kolay.ai. The generated code, while functional in many cases, did not always adhere to best practices for software development, resulting in potential security vulnerabilities, inefficient algorithms, or non-optimal code structures. Debugging the generated code was often more complex than debugging manually written code, as developers needed to understand not only the problem at hand but also the rationale behind the LLM-generated solution. This increased the time spent on code reviews, debugging, and refactoring, which somewhat offset the time savings gained from using LLMs for initial code generation.

Dependency Management

Managing dependencies is a crucial aspect of any software development project, particularly in modular architectures where different components rely on shared libraries or third-party packages. In the Kolay.ai project, the LLM-generated code did not consistently account for dependency management, leading to situations where conflicting versions of packages were suggested for different modules. This lack of consistency in managing dependencies could cause integration failures and required careful manual review to align dependencies across the entire project. The LLM's inability to consider the overall dependency graph highlighted the limitations in its ability to handle complex project-level requirements.

Adapting to Rapid Changes in Requirements

Software development often involves rapidly changing requirements, where functionalities need to be modified or extended based on user feedback or business needs. While LLMs are effective in generating code based on static prompts, they struggle to adapt when requirements evolve dynamically. In the Kolay.ai project, changes in the specifications for certain modules required significant adjustments to the LLM-generated code. Since LLMs do not inherently understand the relationship between evolving requirements and previously generated code, developers had to re-prompt the LLM or manually modify the code to accommodate these changes. This challenge underscored the need for human developers to bridge the gap between LLM-generated output and the real-world dynamics of software development.

Hallucination Problem in LLM-Generated Code

A significant challenge encountered when using LLMs for code generation is the "hallucination" problem, where the model generates code that appears correct but is logically flawed or based on incorrect assumptions. In the context of the Kolay.ai project, hallucinations frequently manifested as the generation of non-existent functions, incorrect method signatures, or references to libraries that were not actually used in the codebase.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

This problem occurred because LLMs, while highly capable of generating syntactically correct code, are not aware of the project's actual state or the specific libraries and functions that have been implemented. As a result, the model might "hallucinate" plausible code that does not align with the reality of the project's existing structure. The hallucination problem complicates the development process because it introduces additional debugging and verification steps. Developers need to carefully review the generated code to identify errors that are not immediately apparent, especially in cases where the code seems valid at first glance. In the Kolay.ai project, this issue led to instances where the LLM-generated code included unnecessary imports, used incorrect variable names, or relied on non-existent classes. Addressing these hallucinations required manual intervention to correct the generated code, ensure that it complied with the existing project structure, and eliminate any superfluous or erroneous components. To mitigate the hallucination problem, it is essential to validate the generated code against the actual project requirements and context. Automated testing and static code analysis tools can help identify discrepancies, but human review remains necessary to ensure that the LLM-generated code performs as intended.

Lack of Memory and Context Retention

Another significant challenge when using LLMs for code generation is the lack of long-term memory and context retention. LLMs operate within a limited context window, meaning they can only "remember" and process a certain amount of information from previous prompts within the same session. Once the context window is exceeded, the model effectively "forgets" earlier parts of the conversation. In the Kolay.ai project, this limitation became problematic when adding new features or making changes to the codebase. As the project evolved and more functionalities were introduced, the LLM would often lose track of previously established conditions, rules, or coding conventions. For example, when adding a new feature that relied on existing models or followed specific business rules, the LLM-generated code would sometimes disregard previous requirements or generate code that conflicted with earlier implementations. This issue arose because the model could not retain the entire history of prompts, project-specific conventions, or dependencies beyond a certain limit. Consequently, developers had to re-prompt the LLM with detailed instructions, including past rules and conditions, to ensure that the generated code aligned with the existing project structure. The lack of memory also made it challenging to maintain consistency across different modules. Each time a new feature was added, the LLM needed to be reminded of the project's architecture, dependencies, and coding standards. This process was time-consuming and increased the risk of introducing errors or inconsistencies into the codebase.

Impact of Hallucination and Memory Limitations on Modular Development

The hallucination problem and lack of memory posed particular challenges for the modular development approach adopted in Kolay.ai. While modular development is advantageous for isolating functionalities and managing dependencies, it also requires that each module adheres to the same overarching design principles and interacts seamlessly with other modules. The hallucination problem often led to situations where LLM-generated modules deviated from the intended design, requiring manual adjustments to ensure compatibility. The lack of memory, meanwhile, affected the ability to integrate new features consistently. For example, if a new module needed to interact with an existing one, the LLM would sometimes generate code that did not take into account shared variables, established naming conventions, or previously defined data structures. This lack of awareness of the broader codebase resulted in additional debugging and manual integration efforts. Developers had to provide detailed prompts repeatedly, including reminders of established project rules, to guide the LLM in generating code that fit within the overall system.

Recommendations for Addressing Hallucination and Memory Issues

To address the challenges of hallucination and lack of memory in LLM-generated code, several strategies can be employed: Use Structured Prompts with Contextual Information: When prompting the LLM, include

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

structured context that reiterates key aspects of the project, such as existing models, functions, and rules. This can help reduce the likelihood of hallucinations and ensure that the LLM generates code that aligns with previously established conditions. Segment the Development Process into Smaller Steps: Break down complex tasks into smaller, manageable prompts that build on each other. By validating the output at each step, developers can catch hallucinations early and reduce the impact of the LLM's memory limitations. Maintain a Project-Specific Code Repository: Create a shared repository of code snippets, templates, and guidelines that can be referenced when generating new modules. This repository can serve as a memory aid, helping to ensure consistency across different LLM-generated outputs. Incorporate Automated Tests and Validation Rules: Use automated testing frameworks to validate the generated code against predefined rules and conditions. This approach can help detect hallucinations and inconsistencies before they are integrated into the project. Implement Manual Code Reviews for Critical Sections: For complex or critical modules, conduct thorough code reviews to ensure that the LLM-generated code meets the project's requirements and adheres to established standards. Manual reviews are especially important for catching subtle hallucinations that automated tools may miss. Iteratively Re-Prompt the LLM with Previous Context: To mitigate the lack of memory, developers can periodically provide a summary of the project's state or past prompts as part of the new prompt. This practice helps re-establish context and guide the LLM in generating code that fits the project's current state.

Results

This section presents a comprehensive evaluation of various LLM technologies used for code generation in the project. The results cover the performance of these models across multiple metrics categorized into intrinsic, extrinsic, and human evaluations, as detailed in Table 2. Each metric provides unique insights into different aspects of the generated code's quality, including its syntactic correctness, functional accuracy, and usability. By analyzing the outcomes from multiple metrics, the study aims to identify both the strengths and limitations of LLM-assisted development. The intrinsic evaluation metrics, such as perplexity and cross-entropy loss, assess the statistical properties of the generated code, providing a baseline understanding of the model's ability to predict code structures. Extrinsic metrics, including functional correctness and compilation rate, evaluate the code's practicality and performance in real-world scenarios. Human evaluation metrics, such as code readability and semantic correctness, offer qualitative insights that are crucial for real-world applications, where maintainability and long-term usability matter. The findings from the comparative analysis reveal patterns in the performance of different LLMs, highlighting areas where some models outperform others. This chapter discusses these outcomes in detail, offering practical recommendations for improving LLM-based code generation and addressing challenges related to consistency, context awareness, and modular integration.

Benefits of Using LLMs for Modular Code Generation

The use of LLMs for modular code generation has brought significant benefits, especially when combined with a modular development approach. The modular approach is essential for managing the complexity of software development, as it allows developers to divide the project into smaller, manageable units. This approach aligns well with the capabilities of LLMs, enabling these models to focus on generating specific functionalities for individual modules without needing to account for the entire project context at once. By breaking down the code generation tasks, developers can more effectively leverage LLMs to generate useful code snippets, accelerating the development process and facilitating rapid prototyping. The necessity of a modular approach becomes evident when considering the limitations of LLMs, such as their lack of long-term memory and limited context retention. Without modularization, LLMs would struggle to handle the interdependencies and complexity of a large-scale project. The modular approach helps mitigate these limitations by enabling LLMs to generate code for discrete tasks, which can later be integrated into a larger system. This incremental process allows for continuous refinement and testing of individual components, making it easier

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

to identify issues and improve the quality of the generated code. Comparing the performance of various LLM technologies across the metrics in Table 1 demonstrates that some models are more effective than others for specific aspects of modular code generation. For example, models with lower perplexity and cross-entropy loss scores tend to produce syntactically correct code, which benefits from the modular approach's ability to isolate issues within smaller code segments. In terms of functional correctness and compilation rate, models that excel at generating structured, context-aware code perform better when tasked with producing individual modules rather than entire systems. This highlights the advantage of using a modular approach to code generation, as it allows developers to exploit the strengths of different LLMs based on the nature of the task. Moreover, the evaluation metrics reveal that while some LLMs perform well in intrinsic measures, such as perplexity and cross-entropy, they may struggle with extrinsic metrics like functional correctness, especially when generating complex modules. This disparity emphasizes the importance of combining modular development with careful human oversight and iterative refinement, ensuring that the LLM-generated code integrates seamlessly into the broader codebase. The modular approach facilitates the incorporation of human feedback, allowing for the refinement of individual modules before integrating them into the overall system. In conclusion, the use of LLMs for modular code generation offers distinct advantages in terms of speed, flexibility, and iterative improvement. The modular approach not only makes it feasible to address the limitations of LLMs but also enables the selective use of different LLM technologies based on their strengths in generating code for specific tasks. This combination of LLM capabilities and modular development is essential for maximizing the benefits of AI-assisted code generation in real-world software projects.

Challenges Faced

The use of various LLMs for modular code generation presented a range of challenges, highlighting the limitations and unique difficulties associated with each tool. While LLM-assisted development offers significant advantages in terms of accelerating the coding process, several common and tool-specific challenges were encountered. The modular approach, while beneficial for isolating tasks, also exposed the limitations of LLMs when generating code for complex, interconnected projects. The following sections detail the key challenges faced across all LLMs, with a focus on the unique strengths and limitations of each tool as listed in Table 1.

Inconsistency in Generated Code

A primary challenge across all tools was inconsistency in the generated code, both in terms of quality and adherence to coding standards. Even when given similar prompts, different LLMs produced code with varying levels of quality, structure, and naming conventions. This inconsistency made it difficult to integrate modules generated by different tools into a cohesive system, requiring manual intervention to standardize the code and ensure uniformity. For example, while GitHub Copilot was able to maintain some consistency within its suggestions, integrating its outputs with code generated by ChatGPT or Tabnine often required additional refactoring due to differences in style.

Hallucinations and Nonexistent Functions

The problem of hallucinations, where LLMs generated plausible-looking but nonexistent functions or libraries, was particularly common with ChatGPT and GitHub Copilot. These tools sometimes created code snippets based on patterns observed in their training data, which led to the generation of functions that were not actually available in the programming language or libraries being used. Addressing hallucinations required thorough code reviews and manual adjustments to replace or remove nonfunctional elements. In contrast, tools like Amazon CodeWhisperer, which have a security focus, generated fewer hallucinated functions but still struggled with function naming consistency.

Lack of Memory and Context Retention

The inability to retain long-term memory and context posed a substantial obstacle for all tools, especially when generating complex, interdependent modules. Tools like ChatGPT and Replit Ghostwriter, which excelled at generating explanations and code from natural language prompts, often failed to maintain con-

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

sistency across multiple sessions. When adding new features or generating code for interconnected modules, these tools frequently disregarded previously established conditions or coding standards, necessitating repeated prompts to maintain alignment with the project's overall structure. GitHub Copilot, although better integrated with IDEs, also struggled with context retention when the project scope extended beyond its immediate suggestions.

Integration Complexity

Integrating code generated by different tools into a unified system presented additional complexities. This challenge was particularly noticeable with Google Codey and Replit Ghostwriter, which are designed for specific use cases or environments. The outputs often required adaptation to fit the broader project architecture. Tools like Amazon CodeWhisperer, which integrate well with AWS services, generated code that was optimized for cloud-based environments but needed modification to function correctly outside these contexts. Manual adjustments were necessary to resolve conflicts, especially when dealing with shared resources like global variables, database connections, or middleware configurations.

Compilation and Execution Failures

Compilation and execution failures were prevalent across most tools. While GitHub Copilot and Tabnine generally produced syntactically correct code, the generated code often failed to execute correctly due to unresolved dependencies or incorrect function calls. ChatGPT, despite its strong natural language understanding, was more prone to generating code with logical errors, as it sometimes misunderstood the problem requirements or used inefficient algorithms. Addressing these failures required developers to spend additional time debugging and ensuring that the generated code adhered to project requirements.

Balancing Code Readability and Functional Correctness

A common challenge when using LLMs was balancing code readability with functional correctness. Although tools like Replit Ghostwriter and Amazon CodeWhisperer excelled at generating readable code, the outputs were sometimes overly verbose or contained redundant logic. On the other hand, tools like Google Codey, which focused more on functional outputs, occasionally generated code that was difficult to read or maintain. Achieving a balance between readability and functional correctness often required manual refactoring to optimize the generated code for long-term maintainability.

Unique Challenges for Each LLM

Each LLM exhibited distinct strengths and limitations, leading to unique challenges that had to be addressed:

- GitHub Copilot: GitHub Copilot demonstrated strong integration with IDEs and generated useful code suggestions across multiple languages. However, it occasionally produced insecure code by default, requiring developers to be vigilant about potential vulnerabilities. Additionally, it struggled with generating context-aware code when multiple interdependent modules were involved, necessitating manual intervention to ensure consistency.
- ChatGPT: ChatGPT excelled in understanding complex natural language prompts and generating versatile code snippets. However, it was prone to hallucinations and often generated incorrect or inefficient code, particularly for tasks requiring deep context awareness. The lack of IDE integration also limited its utility for real-time code completion.
- Tabnine: Tabnine was lightweight and fast, making it ideal for basic code completion tasks. However, it had a limited ability to understand complex context or provide multi-line code generation comparable to more advanced LLMs like ChatGPT or GitHub Copilot. Its suggestions were often limited in scope, requiring developers to expand on the generated code.
- Amazon CodeWhisperer: While Amazon CodeWhisperer offered strong integration with AWS services and focused on generating secure code, it sometimes struggled with prompts that were not specific enough, resulting in generic outputs. Additionally, its language support was not as extensive as GitHub Copilot, which limited its versatility.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

- Google Codey: Google Codey showed strong performance on tasks integrated with Google Cloud and Vertex AI but lacked comprehensive documentation or public information on its capabilities. Its relative newness meant that developers needed to adapt to its unique way of generating code, and there were fewer community resources available for troubleshooting.
- Replit Ghostwriter: Designed with educational and collaborative use cases in mind, Replit Ghostwriter performed well in generating explanations and debugging code. However, it was less suitable for large-scale, production-level code generation due to its focus on educational environments. Integration challenges arose when using Replit Ghostwriter outputs in projects outside the Replit ecosystem.
- Recommendations to Mitigate the Challenges

To address the limitations encountered with each LLM, the following strategies are recommended:

- Use Tool-Specific Strengths: Leverage each LLM's unique capabilities for specific tasks. For example, use Amazon CodeWhisperer for security-focused code generation and GitHub Copilot for seamless IDE integration.
- Iteratively Refine Prompts: Structure prompts to include detailed context and use iterative refinement based on LLM outputs to reduce hallucinations and maintain consistency.
- Employ a Modular Approach: Continue to use a modular development approach to compartmentalize tasks and minimize the impact of context loss or integration issues.
- Combine Human Review with Automated Testing: Ensure that all generated code is subjected to automated testing, followed by manual reviews to address potential logical errors or security concerns.

Discussion of Technologies for the Kolay.ai Project

The Kolay.ai project utilized various LLM technologies to accelerate the development process, automate code generation, and facilitate modular development. The choice of different LLM tools allowed the project team to leverage the unique strengths of each model while addressing their limitations. The modular approach adopted for Kolay.ai proved particularly effective in managing the diversity of tools used, as it enabled the project to compartmentalize tasks and utilize specific LLMs for distinct functionalities. This section discusses the suitability and effectiveness of each LLM technology for the project, highlighting their contributions and limitations. GitHub Copilot was one of the primary tools used for the Kolay.ai project, especially for generating code snippets and completing functions within an integrated development environment (IDE). Its seamless integration with popular IDEs, such as Visual Studio Code, made it a convenient choice for real-time code suggestions and auto-completion. This integration helped maintain a consistent development workflow and enabled rapid prototyping. However, GitHub Copilot occasionally generated insecure code, which required additional review and modification to adhere to security best practices. The tool's limited ability to maintain long-term context across multiple modules also presented a challenge, necessitating frequent manual adjustments to ensure that the generated code aligned with the existing architecture. ChatGPT played a significant role in the Kolay.ai project by generating modular code from natural language prompts and providing detailed code explanations. Its strong natural language understanding made it valuable for translating complex requirements into functional code snippets, particularly for tasks that required precise problem descriptions. Nevertheless, ChatGPT's limitations in IDE integration and context retention became apparent during the project. The tool struggled to generate multi-module code that required an understanding of the broader system. Additionally, the issue of hallucinations—where the model generated plausible but nonexistent functions—was more frequent with ChatGPT compared to other tools. This required additional effort to review and correct the generated code. Tabnine was used for lightweight code completion tasks and whole-line code suggestions. Its fast performance and support for multiple IDEs made it suitable for enhancing productivity in smaller coding tasks, such as filling in simple functions or completing variable assignments. However, compared to LLMs like GitHub Copilot and ChatGPT, Tabnine had limited capabilities for understanding complex project contexts. Its suggestions were often restricted to local code patterns rather than providing more comprehensive, multi-line solutions. The use of Tabnine in Kolay.ai was most effective for low-level code completion, but it was not the primary tool for generating full-function im-

plementations or more intricate modules. Amazon CodeWhisperer brought a security-focused aspect to the Kolay.ai project, especially when generating code for cloud-based components or when working with sensitive data. Its integration with AWS services provided a significant advantage for security scanning and ensuring that cloud-based solutions adhered to best practices. However, CodeWhisperer required more specific prompts to generate high-quality code and had limited language support compared to GitHub Copilot. This constraint meant that while it was effective for AWS-related tasks, it was less versatile for general-purpose code generation in other programming environments used within Kolay.ai. Google Codey was chosen for its strong integration with Google Cloud and Vertex AI, providing an edge in tasks related to data processing and machine learning. The tool performed well in generating code for cloud-native applications and data-centric tasks within the Kolay.ai project. As a relatively new technology, Google Codey had limited public documentation and community support, which occasionally made troubleshooting more challenging. The tool's novelty also meant that there were fewer established best practices for using it effectively, requiring developers to adapt and experiment with different approaches. Replit Ghostwriter was particularly useful for educational and collaborative coding tasks within the Kolay.ai project. It offered debugging assistance and facilitated the explanation of code changes, making it a valuable tool for onboarding new team members or collaboratively working on coding challenges. However, Ghostwriter's primary focus on educational use cases limited its suitability for production-level code generation in the Kolay.ai project. Its integration was optimized for the Replit environment, which posed challenges when integrating its outputs into the broader codebase outside the Replit ecosystem. In the Kolay.ai project, using a variety of LLM technologies allowed the team to exploit the strengths of different tools based on task requirements. GitHub Copilot and Tabnine were advantageous for rapid prototyping and code completion, while ChatGPT excelled in understanding complex natural language prompts and generating modular code. Amazon CodeWhisperer's security features made it ideal for cloud-based components, whereas Google Codey provided a strong foundation for cloud-native and machine learning tasks. Replit Ghostwriter played a complementary role in facilitating code explanations and debugging. However, challenges such as inconsistency in code style, hallucinations, lack of context retention, and integration complexity across tools required a modular development approach. This allowed the Kolay.ai team to compartmentalize tasks and utilize specific LLMs for suitable aspects of the project, while mitigating the limitations through manual reviews and iterative refinement. The Kolay.ai project demonstrates the value of using multiple LLM technologies in a modular software development approach, highlighting the potential of combining AI tools to address various coding needs while acknowledging the necessity for human oversight and adaptive strategies.

DISCUSSION

The Kolay.ai project serves as a comprehensive case study that demonstrates both the potential and limitations of using LLMs for modular software development. By leveraging multiple LLM tools such as GitHub Copilot, ChatGPT, Tabnine, Amazon CodeWhisperer, Google Codey, and Replit Ghostwriter, the development process was significantly accelerated, enabling rapid prototyping and automation of routine coding tasks. This hybrid approach capitalized on the unique strengths of each tool, allowing the team to manage the complexities of building a large-scale software platform. However, the project also uncovered significant challenges that illustrate the current limitations of LLM technologies in real-world software development.

Key Benefits of Using LLMs in Modular Software Development

The results from the Kolay.ai project highlight the benefits of adopting a modular development approach when using LLMs for code generation. By breaking down the project into smaller, manageable modules, the team could more easily utilize LLMs for generating discrete code segments. This modular approach allowed the team to:**Accelerate Development Cycles** : LLMs helped to quickly generate initial versions of code for each module, significantly speeding up the development timeline. This was especially beneficial in tasks like auto-completion, function generation, and translating requirements into code.**Isolate Complexities**

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

: Dividing the project into independent modules allowed for focused problem-solving and testing, making it easier to identify and fix issues within individual components before integration. This approach also helped in managing the complexity of the project, as LLMs could be applied to specific functionalities without overwhelming them with the entire system's context.**Iteratively Refine and Test Code** : The modular structure facilitated iterative refinement, where individual modules could be tested, reviewed, and enhanced before being combined into the complete system. This helped to reduce the impact of issues like hallucinations, inconsistencies, and integration conflicts.

Challenges Faced in the Kolay.ai Project

While LLMs brought significant benefits, the Kolay.ai project also revealed several critical challenges:**Inconsistency in Code Quality and Hallucinations** : Despite their ability to generate syntactically correct code, LLMs often produced inconsistent results. Tools like ChatGPT and GitHub Copilot sometimes generated plausible but incorrect or nonexistent functions, leading to additional debugging and code review efforts.**Lack of Long-Term Memory and Context Retention** : The LLMs struggled to retain project-wide context across multiple prompts. This limitation required repeated prompts to remind the models of established conditions, dependencies, or coding standards. The lack of context retention was especially problematic when generating interconnected modules that required consistent naming conventions, shared resources, or architectural patterns.**Integration Complexity** : Combining code generated by different LLMs introduced integration challenges, as each tool had its coding style, structure, and error patterns. The team had to manually reconcile these differences to ensure the final integrated system worked seamlessly. This process included resolving conflicts with shared resources, adjusting middleware configurations, and aligning variable naming conventions.**Balancing Functional Correctness with Readability** : Achieving a balance between functional correctness and code readability was another significant challenge. Some tools generated complex, verbose code that was difficult to maintain, while others produced code that was concise but lacked robustness. This required additional manual refactoring to optimize the generated code for both maintainability and functionality.

Unique Strengths and Limitations of the LLM Tools Used

Each LLM used in the project brought unique strengths, but also presented specific limitations:

- GitHub Copilot: Its integration with IDEs made it effective for real-time code suggestions and rapid prototyping. However, it occasionally generated insecure code and struggled with long-term context retention.
- ChatGPT: ChatGPT excelled in understanding natural language prompts and translating them into code, but it was prone to hallucinations and lacked IDE integration, making real-time development more challenging.
- Tabnine: This tool provided fast code completion for smaller tasks, but had limited capabilities for generating multi-line solutions or understanding complex project requirements.
- Amazon CodeWhisperer: Its security-focused code generation and AWS integration were beneficial for cloud-based tasks. However, it required specific prompts to produce high-quality outputs and had limited language support.
- Google Codey: Google Codey performed well for cloud-native applications but lacked comprehensive community resources and documentation, making troubleshooting more difficult.
- Replit Ghostwriter: Suitable for educational and collaborative environments, Ghostwriter was less appropriate for production-level code generation, limiting its use in large-scale tasks.

Implications for the Literature and Future Research

The Kolay.ai project contributes to the ongoing discourse on AI-assisted software development by providing practical insights into the use of LLMs for modular software generation. Unlike most existing studies that

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54826063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

focus on generating isolated code snippets, this project demonstrated the applicability of LLMs in developing complex, real-world systems using a modular approach. The results emphasize that while LLMs can significantly enhance productivity and speed up the development process, they cannot yet replace the need for human oversight, especially when dealing with complex dependencies, security requirements, or long-term project consistency. Future research should aim to address the limitations uncovered in this study. Potential areas for further investigation include:**Improving Context Retention in LLMs** : Developing mechanisms for LLMs to better retain long-term context and project-wide dependencies across multiple prompts could significantly reduce the need for repeated instructions and manual intervention.**Reducing Hallucinations and Enhancing Functional Accuracy** : Strategies to minimize the occurrence of hallucinations and improve the accuracy of generated functions are essential for making LLMs more reliable in practical applications.**Exploring Hybrid AI-Human Development Models** : Further research could investigate how LLMs can be integrated into hybrid development workflows where AI-generated code complements human expertise, leading to more efficient and robust software engineering practices.

Recommendations for Future LLM-Assisted Projects

Based on the findings from the Kolay.ai project, the following recommendations can help optimize LLM-assisted development workflows:**Continue to Use a Modular Approach** : Modular development helps compartmentalize tasks, making it easier to manage integration complexities, perform testing, and ensure consistency across modules.**Leverage the Unique Capabilities of Different LLMs** : Utilize LLM tools according to their strengths—such as using GitHub Copilot for rapid code completion, ChatGPT for translating complex requirements, and Amazon CodeWhisperer for cloud-specific tasks.**Combine Automated and Manual Code Reviews** : Automated testing should be complemented with thorough manual code reviews to identify logical errors, security vulnerabilities, and readability issues in the generated code.**Implement Structured Prompting Techniques** : To reduce hallucinations and improve output consistency, prompts should be structured to include specific details about the project's architecture, dependencies, and coding standards. Table 3 presents statistical data derived from the Kolay.ai project. The statistics cover aspects such as the number of modules, AI-driven features, data visualization functionalities, and user management capabilities. Table 3: Evaluation Metrics of Kolay.AI

Metric	Value	Description
Number of functional modules	80+	Total number of distinct functionalities defined in urls.py.
AI-driven features	10+	Includes RFM analysis, sales predictions, and recommendations.
Data visualization functionalities	5	Heatmap, sankey diagrams, sales-time charts, treemaps, and boxplots
User management-related functionalities	10+	Covers user registration, role management, password resets, etc.
Number of URL endpoints	85	Unique URL endpoints mapped to various views.
Total database models	20+	Includes models for customer, product, offer, company, etc.
CRUD operations covered	100%	Supports Create, Read, Update, Delete across modules.
Average module code length (lines)	50-100	Code complexity across different modules.
Average response time for key queries	<200 ms	Measured response time for fetching customer and product data.
Maximum concurrent users tested	50	Tested under simulated load for system performance.

The statistics reveal the extent and complexity of the project, demonstrating a balanced development approach that covers a wide range of functionalities essential for business operations. The integration of AI-driven modules enables automated decision-making and predictive analytics, adding significant value to the platform.

Success of LLMs in the Project

The use of LLMs, especially GPT-based models, played a crucial role in several aspects of the Kolay.ai project:

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

Automating Repetitive Coding Tasks :

LLMs assisted in generating boilerplate code for CRUD operations, reducing development time by approximately 40%. The automated code generation was particularly effective for creating Django views, serializers, and forms.

Example: The code for customer and product management views was initially generated using LLMs and then manually refined.

Improving Natural Language Understanding :

The integration of LLMs facilitated the implementation of user-friendly interfaces where users could query the system in natural language. This was particularly useful for generating sales forecasts and performing RFM analysis.

Enhanced Debugging and Testing :

LLMs were used to automate test case generation, identifying edge cases in the code. This improved the platform's robustness, reducing the bug count by approximately 30% compared to traditional manual testing.

Accelerated Documentation and Code Comments :

Documentation and inline code comments were generated using LLMs, which improved the readability and maintainability of the codebase. This also led to a 20% reduction in onboarding time for new developers.

Table 4 illustrates the quantitative impact of LLMs on the project:

Table 4: Impact of LMMs on the Project

Task	Traditional Approach	LLM-Assisted Approach	Improvement (%
Code generation time (hours)	200	120	40% reduction
Bug count during initial testing	50	35	30% reduction
Test case coverage	70%	90%	20% increase
Documentation completeness score	7/10	9/10	20% improvement
Average time for new developer onboarding	10 days	8 days	20% reduction

The project's success, in terms of LLM integration, can be evaluated across multiple dimensions:

Accuracy and Completeness :

LLMs successfully generated code snippets that met 85-90% of the functional requirements, needing only minimal modifications for edge cases and custom logic.

Efficiency Gains :

The use of LLMs led to substantial efficiency gains, with a reduction in development time by 30-40%. This gain was especially noticeable in repetitive tasks such as generating views and serializers in Django.

Code Quality and Maintainability :

The quality of the code improved with the consistent use of LLM-generated documentation and comments. Automated test generation contributed to higher test coverage, leading to a more reliable system.

Scalability and Adaptability :

The modular design facilitated by LLMs allowed for rapid integration of new features, scaling the platform's capabilities without significant architectural changes.

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54820063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.

ConclusIon

In conclusion, the Kolay.ai project demonstrated the transformative potential of LLMs in modular software development while highlighting the need for structured approaches to manage the limitations of AI-generated code. The findings from this study provide a roadmap for future LLM-assisted development, emphasizing the importance of modularization, hybrid development models, and human oversight to fully leverage AI's capabilities. The lessons learned from Kolay.ai contribute valuable knowledge to the field of AI-driven software engineering, setting the stage for future advancements in LLM technology and its application in complex software development projects. The creation of Kolay.ai using LLM-generated code offers valuable insights into the potential and limitations of AI-assisted software development. While LLMs can significantly speed up the development process and facilitate modular design, human intervention remains critical for refining and integrating the generated code into a cohesive project. The lessons learned from this experience contribute to the ongoing discourse on the role of AI in software engineering and provide practical guidance for developers seeking to leverage LLMs for modular software development. This study is significant because it illustrates the transformative impact of LLMs on the software development process in real-world business applications. As enterprises seek to adopt AI-powered solutions to streamline their operations, understanding the practical benefits and limitations of LLMs is crucial. This research contributes to the growing body of knowledge on AI-driven development methodologies and serves as a guiding framework for other projects aiming to leverage LLMs for efficiency and innovation. By providing empirical evidence from the Kolay.ai project, this study lays the groundwork for future advancements in AI integration, ensuring that developers can effectively harness the power of LLMs to solve complex business challenges. While the case study of Kolay.ai provides valuable insights into the implementation and benefits of LLMs in software development, future research can explore several directions to further enhance the impact of AI-driven methodologies. One potential area is the optimization of LLM models for domain-specific applications, where custom fine-tuning could lead to even greater performance improvements. Additionally, expanding the evaluation framework to include user feedback and business outcome metrics would provide a more holistic view of the impact. Future studies could also investigate the integration of LLMs with other emerging technologies, such as reinforcement learning or automated machine learning (AutoML), to create a more robust and adaptive software development pipeline. By addressing these areas, future research can continue to push the boundaries of what AI-driven solutions can achieve in business intelligence and software engineering. REFERENCES Amazon Web Services, Inc. (n.d.). *Amazon CodeWhisperer* . Retrieved October 10, 2024, from https://aws.amazon.com/codewhisperer/ Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization (pp. 65-72). Bang, Y., Cahyawijaya, S., Lee, N., Lim, W., & Suh, B. (2023). A comprehensive survey of AI-powered code generation tools. *ACM Computing Surveys, 55* (12), 1-38. Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. Journal of machine learning research, 3(Feb), 1137-1155. Buse, R. P. L., & Weimer, W. (2010). Learning a metric for code readability. *IEEE Transactions on Software Engineering* , *36* (4), 546-558. C. Larman and V. R. Basili, "Iterative and Incremental Development: A Brief History," *IEEE Computer* , vol. 36, no. 6, pp. 47-56, June 2003. doi: 10.1109/MC.2003.1204375. Chen, L., & Monperrus, M. (2019). A literature study of embeddings on source code. *ACM Computing Surveys (CSUR)* , *51* (3), 1-27. Chen, X., Liu, C., & Song, D. (2021). Tree-based deep model for program repair. In *Proceedings of the 2021 International Conference on Software Engineering* (pp. 335-346). Copeland, L. (2004). *A Practitioner's Guide to Software Test Design* . Artech House. D. L. Parnas, "On the Criteria to Be Used in Decomposing Systems into Modules," Commun. ACM, vol. 15, no. 12, pp. 1053-1058, Dec. 1972. doi: 10.1145/361598.361623. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software* . Reading, MA: Addison-Wesley, 1995. Gershgorn, D. (2021, June 29). GitHub and OpenAI launch a new AI tool that generates its own code. *The Verge* . Retrieved October 10, 2024, from [invalid URL removed] Google. (n.d.). *Google Codey* . Retrieved October 10, 2024 Hobbs, J. R. (1979). Coherence and coreference. Cognitive science, 3(1), 67-90. Howcroft, D. M., Lewis, D., & Dennison, R. (2013). The role of metrics in automating text generation. *Proceedings of the Annual*

Meeting of the Association for Computational Linguistics . Howcroft, D. M., Lewis, R. L., & Dennison, M. (2013). Evaluating the fluency of text with a crowdsourced fluency task. In Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 831-835). Jelinek, F., Mercer, R. L., Bahl, L. R., & Baker, J. K. (1977). Perplexity—a measure of the difficulty of speech recognition tasks. The Journal of the Acoustical Society of America, 62(S1), S63-S63. Lin, C. Y. (2004). Rouge: A package for automatic evaluation of summaries. In Text summarization branches out (pp. 74-81). Manning, C. D., & Schütze, H. (1999). Foundations of statistical natural language processing. MIT press. Mendonça, M., & Faria, J. P. (2011). Evaluating functional correctness of systems through manual testing: Evidence from industry. *International Symposium on Software Reliability Engineering* . Nguyen, A., Nguyen, T., & Nguyen, H. (2013). Learning API usages from bytecode: A statistical approach. *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering* . OpenAI. (2022). *ChatGPT* . Retrieved October 10, 2024, from https://openai.com/blog/chatgpt/ Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). Bleu: a method for automatic evaluation of machine translation. In Proceedings of the 40th annual meeting of the Association for Computational Linguistics (pp. 311-318). R. S. Pressman, *Software Engineering: A Practitioner's Approach* , 8th ed. New York, NY: McGraw-Hill, 2014. Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. arXiv preprint arXiv:1606.05250. Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing* . Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. arXiv preprint arXiv:1606.05250. Raychev, V., Bielik, P., & Vechev, M. (2016). Probabilistic model for code with decision trees. In Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and *Applications* (pp. 731-747). Replit. (n.d.). *Replit Ghostwriter* . Retrieved October 10, 2024, from https://replit.com/site/ghostwriter T. Ahmad, Q. Zhang, and X. Liu, "The Challenges of Using AI for Automated Code Generation: A Systematic Review," *Software Pract. Exp.* , vol. 52, no. 7, pp. 1450-1465, July 2022. doi: 10.1002/spe.3012. Tufano, M., Pantiuchina, J., Watson, C., Bavota, G., & Poshyvanyk, D. (2018). On learning meaningful code changes via neural machine translation. *Proceedings of the 40th International Conference on Software Engineering* . Voorhees, E. M., & Tice, D. M. (2000). Building a question answering test collection. In Proceedings of the 23rd annual international ACM SIGIR conference on Research and development in information retrieval (pp. 200-207). Voorhees, E. M., & Tice, D. M. (2000). The TREC-8 question answering track evaluation. *Text REtrieval Conference* .

Posted on 12 Oct 2024 | The copyright holder is the author/funder. All rights reserved. No reuse without permission. | https://doi.org/10.22541/au.172871465.54829063/v1 | This is a preprint and has not been peer-reviewed. Data may be preliminary.